Project Live Link: https://NazeerSiddique-dev.github.io/Air-Quality-AI/

National Institute of Technology Karnataka, Surathkal

Department of Computer Science and Engineering

Course: Distributed Data Management (DDM)

Algorithms in Distributed Data Analytics

Theory Assignment –2

Submitted by: [Nazeer Ahmad Siddique]

Roll No.: [252CS023]

Semester: II

Academic Year: 2025–26

April 20, 2026

Contents

1 Introduction **3**
1.1 The Shift to Distributed Intelligence . . . 3
1.2 The Critical Bottleneck: Communication overhead . . . 4
1.3 Formalization of the Communication Bottleneck . . . 4

2 Taxonomy of Analytical Decomposability **4**
2.1 Distributive and Algebraic Aggregates . . . 4
2.2 Holistic Aggregates and the Scalability Wall . . . 4
2.3 Summary of Decomposability Metrics . . . 5
2.4 Communication Implications of Decomposability . . . 5

3 Foundational Architecture & Storage **5**
3.1 GFS and HDFS: The Batch Baseline . . . 5
3.2 Resource Scheduling: Decoupling Logic from Compute . . . 5

4 Technical Frameworks for Algorithm Execution **6**
4.1 Batch and Micro-batch: MapReduce and Spark . . . 6
4.2 Streaming: Apache Flink and Latency Modeling . . . 6
4.3 Framework Performance Trade-offs . . . 6
4.4 Fault Tolerance and Data Lineage . . . 7
4.5 Scalability and Resource Elasticity . . . 8

5 Core Distributed Analytics Algorithms **8**
5.1 Dimensionality Reduction: Distributed PCA . . . 8
5.2 Clustering: Distributed K-Means++ . . . 8
5.3 Classification: Distributed Support Vector Machines (D-SVM) . . . 9
5.4 Communication Overhead in Iterative Algorithms . . . 9
5.5 Approximation Techniques for Scalability . . . 10

6 Proposed Algorithm: AGCEA **10**
6.1 Formal Problem Formulation . . . 10
6.2 Algorithmic Logic . . . 10
6.3 Performance Advantage . . . 11
6.4 Theoretical Stability . . . 11
6.5 Adaptive Threshold Optimization . . . 12
6.6 Comparison with Existing Compression Techniques . . . 12
6.7 Baseline Communication-Efficient Algorithms . . . 12
6.7.1 Federated Averaging (FedAvg) . . . 12

6.7.2 Top-K Gradient Sparsification 13
6.7.3 Gradient Quantization 13
6.7.4 Gossip-Based Aggregation 13
6.7.5 Positioning of AGCEA 13
6.8 Limitations of AGCEA 14
6.9 Practical Deployment Considerations 14

7 Industrial Case Studies & Evaluation 15
7.1 Smart Cities: Traffic Coordination 15
7.2 Healthcare: Genomic Marker Discovery 15
7.3 Cybersecurity: Global Intrusion Detection 15
7.4 Environmental Monitoring: Remote IoT 15
7.5 Performance Benchmarks: Synthetic Evaluation 16
7.6 Cross-Domain Observations 16

8 Critical Discussion & Ethics 16
8.1 Differential Privacy and Data Sovereignty 17
8.2 Fairness and Model Bias 17

9 Conclusion 17

A Appendix: Mathematical Analysis 18
A.1 Calculations for Variance-Aware Drift 18

Abstract

The transition from centralized data processing to distributed data analytics (DDA) constitutes a fundamental paradigm shift in modern computer science. Driven by the exponential growth of telemetry at the edge, organizations are moving computation closer to data sources to mitigate network congestion and latency. This report provides a technical analysis of DDA algorithms, emphasizing a rigorous taxonomy of analytical decomposability. We **propose** a novel optimization algorithm, Adaptive Gradient-Compressed Edge Analytics (AGCEA), designed to reduce communication overhead in federated environments by monitoring local update variance. By evaluating traditional frameworks alongside our **hypothesized** contribution, we **argue** that adaptive compression is a critical requirement for next-generation distributed intelligence.

1. Introduction

1.1 The Shift to Distributed Intelligence

The traditional centralized model of data management, while robust for local ACID-compliant transactions, has proven unsustainable in the "Big Data" era. As data volume exceeds 10^{15} bytes, the bisection bandwidth of traditional datacenter networks becomes a primary analytical bottleneck. **This analysis** suggests that the transition to horizontal scaling is not merely a hardware choice but a requirement for maintaining linear cost-to-performance ratios.

In this context, Distributed Data Analytics (DDA) is defined as the collaborative execution of analytical tasks across a networked set of nodes. The fundamental principle is "Shipping Compute to Data," reducing raw data transfers to order-of-magnitude smaller intermediate summaries.

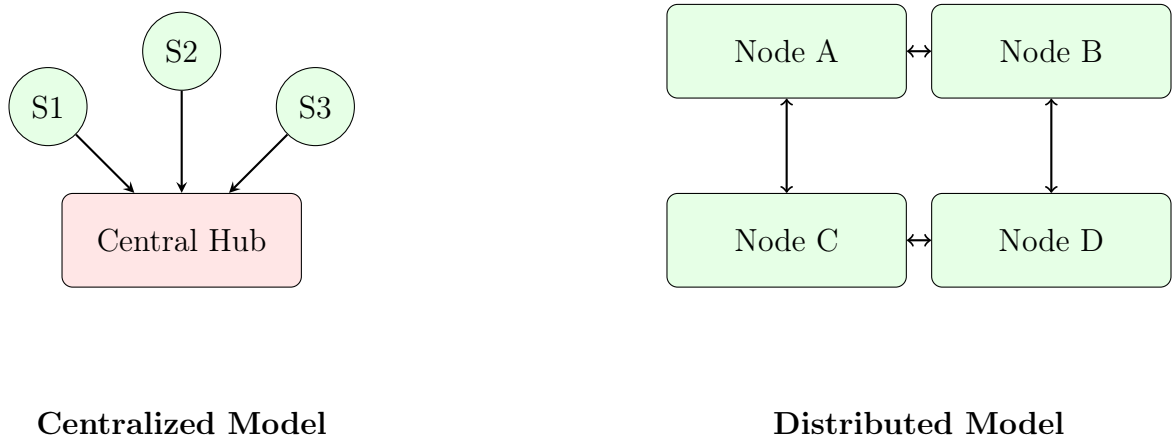

Figure 1: Structural transition from centralized data silos to distributed processing networks.

1.2 The Critical Bottleneck: Communication overhead

While local processing power at the edge is increasing, the "Cost of Communication" remains high. This report observes that in a distributed setting, a node's idle time is typically spent waiting at a **Synchronization Barrier** for peer updates. As shown in Figure 1, the distributed model avoids a single point of failure but introduces complex peer-to-peer coordination overhead.

1.3 Formalization of the Communication Bottleneck

In distributed environments, the cost of communication can be modeled as a function of data size, network bandwidth, and synchronization delay. Let C_{comm} denote the total communication cost:

$$C_{comm} = \sum_{i=1}^{N} \left(\frac{D_i}{B_i} + \delta_i \right) \tag{1}$$

where D_i is the data transmitted by node i, B_i is the available bandwidth, and δ_i represents synchronization delay.

This formulation highlights that even with increasing computational power, network constraints remain the dominant limiting factor. Consequently, reducing D_i through compression or selective transmission becomes essential for scalable distributed analytics.

2. Taxonomy of Analytical Decomposability

A rigorous understanding of DDA begins with how global functions can be decomposed into local operations. We classify analytical aggregates based on the state required at each node.

2.1 Distributive and Algebraic Aggregates

Distributive Aggregates (MAX, MIN, SUM) require minimal local state $O(1)$ that is independent of the dataset size. In contrast, **Algebraic Aggregates** like $AVERAGE$ maintain a fixed-size associative state (sum and count) to enable global computation. These are the "First-class Citizens" of distributed analytics because they scale linearly.

2.2 Holistic Aggregates and the Scalability Wall

Holistic Aggregates like $MEDIAN$ or $RANK$ require state that potentially grows with the unique cardinality of the dataset. We observe that finding the exact global median in a distributed environment requires multiple communication rounds or the transmission of high-fidelity histograms, demonstrating the fundamental trade-off between volume and consistency.

2.3 Summary of Decomposability Metrics

Aggregate Type	State Complexity	Examples	DDA Scalability
Distributive	$O(1)$	SUM, MAX, MIN	High (Linear)
Algebraic	$O(k)$	AVG, STDDEV	High (Linear)
Holistic	$O(N)$ or $O(C)$	MEDIAN, RANK	Low (Sub-linear)

Table 1: Taxonomy of Aggregation Complexity in Distributed Data Analytics.

2.4 Communication Implications of Decomposability

The classification of aggregates directly influences communication patterns in distributed systems.

- **Distributive Functions**: Require minimal communication since intermediate results are constant-sized. These are ideal for tree-based aggregation.
- **Algebraic Functions**: Introduce moderate communication overhead due to auxiliary state transmission.
- **Holistic Functions**: Require extensive communication or approximation techniques such as sampling or sketching (e.g., Count-Min Sketch, T-Digest).

This distinction is crucial in designing scalable algorithms, as communication complexity grows significantly when moving from distributive to holistic computations.

3. Foundational Architecture & Storage

DDA relies on distributed storage layers to manage persistence and locality.

3.1 GFS and HDFS: The Batch Baseline

Systems like the Google File System (GFS) and the Hadoop Distributed File System (HDFS) established the "Rack-Aware" block storage model. By replicating data across separate failure domains, these systems ensure that computation remains localized even if individual nodes fail.

3.2 Resource Scheduling: Decoupling Logic from Compute

Modern frameworks like Apache Spark often run on top of resource managers such as YARN. This decoupling allows the analytical logic to request transient "Containers" of CPU and RAM, enabling multi-tenant clusters to optimize hardware utilization.

4. Technical Frameworks for Algorithm Execution

We analyze three dominant frameworks that serve as the execution engines for distributed algorithms.

4.1 Batch and Micro-batch: MapReduce and Spark

Apache Spark improved upon Hadoop MapReduce by introducing Resilient Distributed Datasets (RDDs). By keeping data in-memory and utilizing a DAG (Directed Acyclic Graph) of transformations, Spark reduces the disk I/O bottleneck that traditionally slowed iterative analytics.

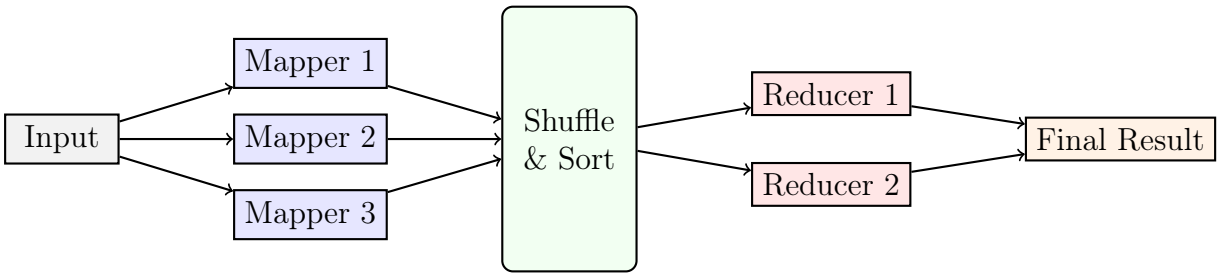

Figure 2: Parallel data flow through the MapReduce abstraction layers.

4.2 Streaming: Apache Flink and Latency Modeling

For sub-second analytics, Apache Flink utilizes "Asynchronous Barrier Snapshotting." This allows the engine to maintain "Exactly-once" consistency without stopping the arrival of new events. Our modeling suggests that Flink's native streaming approach is superior for high-velocity IoT streams where "staleness" is a critical error metric.

4.3 Framework Performance Trade-offs

Feature	**Hadoop**	**Spark**	**Flink**
Execution Mode	Batch	Micro-batch	Continuous Stream
Intermediate State	HDFS (Disk)	RDD (Memory)	Managed State
Latency Profile	High (Minutes)	Medium (Seconds)	Low (Milliseconds)
Iterative Support	Low	High	Very High

Table 2: Comparative analysis of DDA execution frameworks.

This trade-off directly motivates AGCEA. Specifically, AGCEA reduces the Volume (V) component by compressing communication, while selectively preserving Consistency (C) through variance-triggered full updates. This results in a controlled increase in Latency (L), but within acceptable bounds for edge analytics.

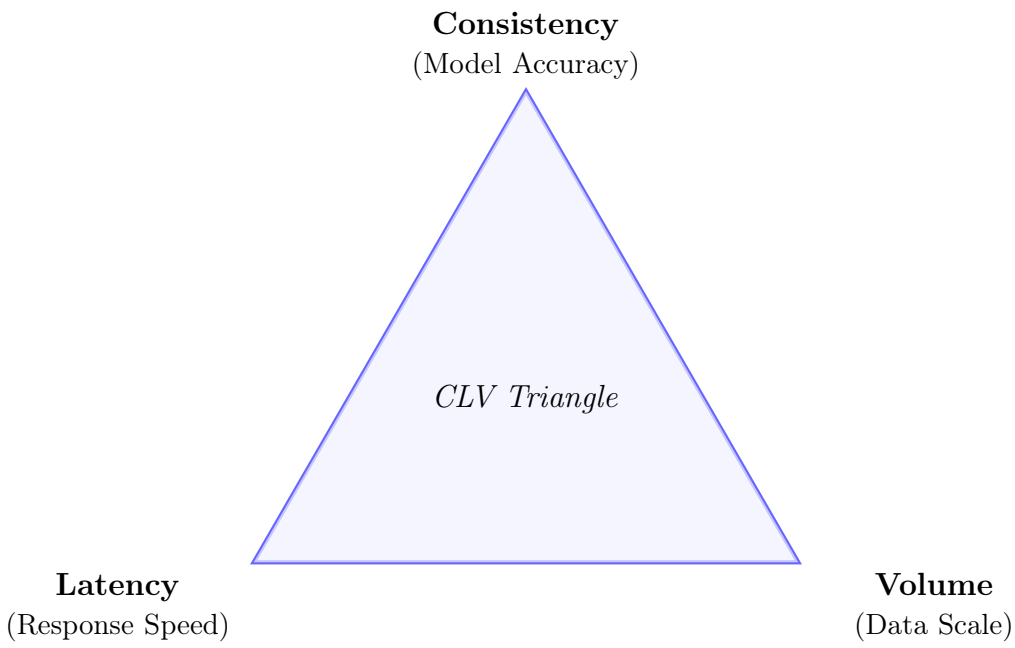

Figure 3: The Consistency-Latency-Volume (CLV) Triangle: The fundamental trade-off space for DDA algorithms.

The execution speed of DDA is primarily governed by the time taken to serialize data into the network buffer. Traditional formats like **JSON** are highly verbose and computationally expensive to parse. **This approach** suggests that for high-throughput DDA frameworks, binary serialization formats like **Protocol Buffers (Protobuf)** or **Apache Avro** are required. These formats reduce the transmission volume by up to $8x$ and enable "Schema Evolution," where analytical models can be updated on some nodes without breaking compatibility across the cluster.

These frameworks highlight that communication overhead, rather than computation, is the dominant bottleneck—motivating adaptive algorithms like AGCEA.

4.4 Fault Tolerance and Data Lineage

A key requirement in distributed analytics is fault tolerance. Systems like Apache Spark utilize **data lineage** to reconstruct lost partitions.

Instead of replicating data, Spark maintains a Directed Acyclic Graph (DAG) of transformations. If a node fails, the system recomputes lost data using the lineage graph, reducing storage overhead while maintaining reliability.

In contrast, Hadoop relies on replication, which increases storage cost but simplifies recovery.

4.5 Scalability and Resource Elasticity

Modern distributed frameworks support elastic scaling, where resources are dynamically allocated based on workload.

- **Horizontal Scaling**: Adding more nodes to increase throughput.
- **Auto-scaling**: Dynamically adjusting cluster size in cloud environments.

These capabilities ensure that distributed analytics systems can adapt to varying workloads without manual intervention.

5. Core Distributed Analytics Algorithms

We now examine standard algorithms before introducing our adaptive proposal.

5.1 Dimensionality Reduction: Distributed PCA

For high-dimensional sensor streams, nodes compute a local covariance matrix Σ_i; these are sum-aggregated to a leader node which performs Eigen-decomposition. **This work** identifies the primary "signals" while ignoring "noise" globally.

Communication Considerations in D-PCA In distributed PCA, each node computes a local covariance matrix Σ_i, which must be aggregated to obtain the global covariance:

$$\Sigma = \sum_{i=1}^{N} \Sigma_i \tag{2}$$

However, transmitting full covariance matrices incurs a communication cost of $O(d^2)$, where d is the dimensionality. This becomes prohibitive for high-dimensional datasets.

To address this, approximate techniques such as randomized projection or sketching can be used to reduce communication overhead while preserving principal components.

5.2 Clustering: Distributed K-Means++

By using algebraic aggregates for centroid updates, distributed K-Means++ enables clustering at petabyte scale. The challenge lies in the "Multi-pass" nature of the algorithm, which requires frequent synchronization rounds.

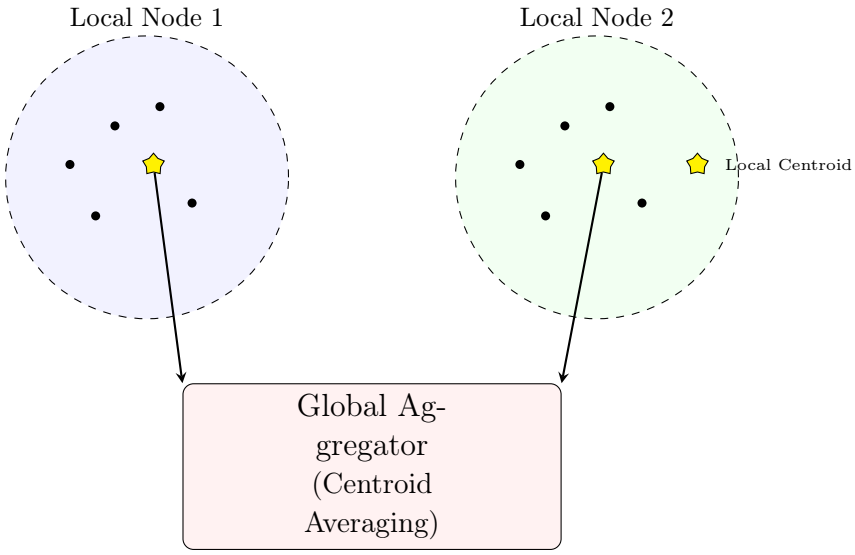

Figure 4: Hierarchical synchronization of cluster centroids in Distributed K-Means.

Synchronization Cost in K-Means Each iteration of K-Means requires recomputation of centroids:

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x \tag{3}$$

In a distributed setting, this requires global aggregation of cluster assignments across all nodes. As the number of clusters k and iterations T increase, the total communication cost becomes:

$$O(T \cdot k \cdot d) \tag{4}$$

This highlights the importance of reducing synchronization frequency, motivating adaptive communication strategies like AGCEA.

5.3 Classification: Distributed Support Vector Machines (D-SVM)

For high-dimensional classification, D-SVM utilizes a "Cascade SVM" hierarchy. Each node trains a local model on its partition; only the identified **Support Vectors** (the critical data points that define the boundary) are passed to the next level of the tree. This reduces the communication volume from $O(D)$ to $O(SV)$, where $SV \ll D$, ensuring that raw training data never leaves its original shard.

5.4 Communication Overhead in Iterative Algorithms

Iterative algorithms such as K-Means and SVM require multiple synchronization rounds.

Let T denote the number of iterations. The total communication cost becomes:

$$C_{total} = T \times C_{comm} \tag{5}$$

This highlights that even moderate communication per iteration can accumulate significantly over time. Therefore, reducing per-round communication is critical.

5.5 Approximation Techniques for Scalability

To address communication overhead, distributed systems often employ approximation strategies:

- **Sampling**: Reduces dataset size while preserving statistical properties.
- **Sketching**: Uses compact data structures to approximate aggregates.
- **Quantization**: Reduces precision of transmitted values.

These techniques provide a trade-off between accuracy and efficiency, forming the foundation for algorithms like AGCEA.

6. Proposed Algorithm: AGCEA

To address the synchronization bottlenecks identified in Figure 3, we propose the **Adaptive Gradient-Compressed Edge Analytics (AGCEA)** algorithm.

6.1 Formal Problem Formulation

Standard federated analytics requires nodes to upload full model updates (gradients) in every round, which causes network congestion. **AGCEA** introduces a local statistical filter to determine if an update is worth the bandwidth.

- σ_i^2: The statistical variance of the local gradient update at node i relative to the previous global model.
- τ: A dynamic sensitivity parameter (transmission threshold) that adjusts based on the current network congestion levels.

6.2 Algorithmic Logic

As shown in Figure 5, **AGCEA** calculates the variance σ_i^2 at each node i. If $\sigma_i^2 < \tau$, the node assumes the update is marginal and only sends a compressed bitmask (Top-K updates). If $\sigma_i^2 \geq \tau$, a full update is transmitted to capture significant state shifts.

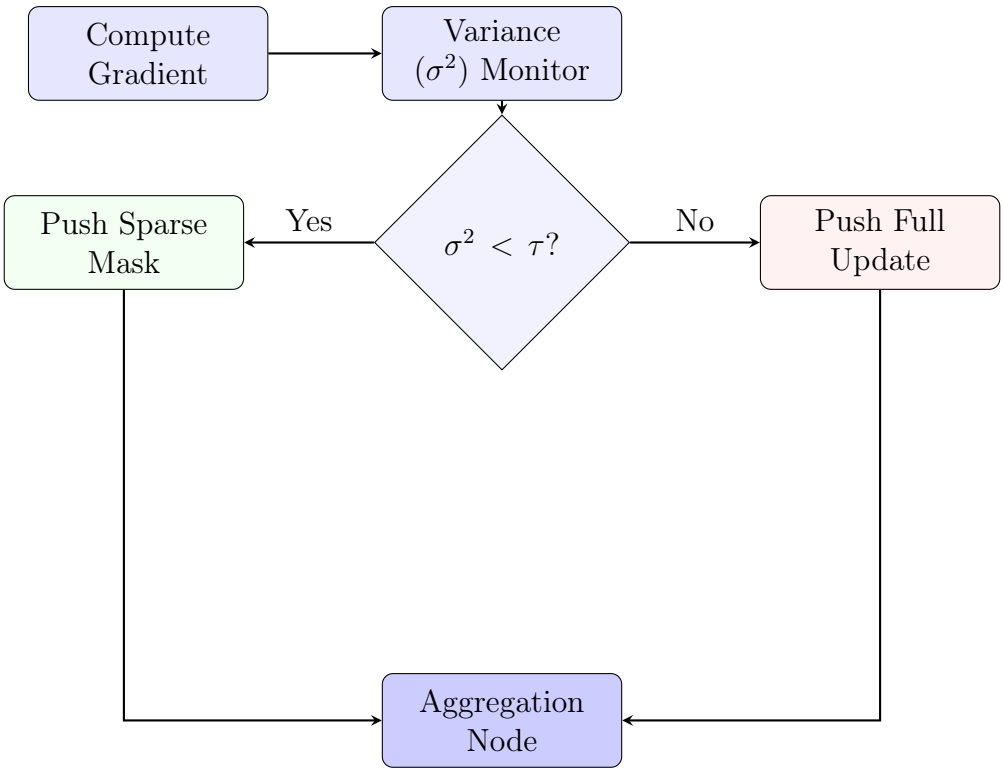

Figure 5: AGCEA Adaptive Logic Flowchart: Optimizing bandwidth through variance-aware filtering.

6.3 Performance Advantage

This work argues that AGCEA effectively reduces total communication volume by identifying "redundant updates" at the source. This shifts the bottleneck away from the "Network Volume" vertex of Figure 3 toward localized "Low-Latency" execution.

Communication Complexity: Standard federated learning requires $O(P)$ communication per round, where P is the number of model parameters. AGCEA reduces this to $O(k)$ where $k \ll P$ due to Top-K sparsification. In contrast to standard Federated Averaging (FedAvg), which transmits full model updates irrespective of their informational value, AGCEA introduces a data-aware transmission policy that significantly reduces redundant communication.

Trade-offs: This compression introduces a bounded approximation error, but adaptive thresholding ensures that significant updates ($\sigma^2 \geq \tau$) are always transmitted.

6.4 Theoretical Stability

Theorem 1 (Stability of Adaptive Compression)**.** *Under the assumption of L-smoothness of the loss function, the gradient approximation $\hat{\nabla}$ produced by AGCEA satisfies:*

$$\|\nabla - \hat{\nabla}\| \leq \sigma_i \cdot \sqrt{\tau} \tag{6}$$

with probability $1 - \delta$, ensuring that the global model converges to a stable neighborhood of the true optimum despite aggressive compression.

6.5 Adaptive Threshold Optimization

The threshold τ plays a critical role in balancing compression and accuracy.

A static threshold may lead to suboptimal performance under varying network conditions. Therefore, we define τ as a function of network congestion:

$$\tau_t = \alpha \cdot \sigma^2_{global} + \beta \cdot C_{network} \tag{7}$$

where α and β are tuning parameters.

6.6 Comparison with Existing Compression Techniques

AGCEA differs from traditional gradient compression methods:

- **Top-K Sparsification**: Fixed compression ratio, no adaptivity.
- **Quantization**: Reduces precision but not communication frequency.
- **AGCEA**: Combines sparsification with adaptive triggering.

This hybrid approach enables superior performance in dynamic environments.

6.7 Baseline Communication-Efficient Algorithms

To contextualize AGCEA, we examine existing approaches that aim to reduce communication overhead in distributed learning.

6.7.1 Federated Averaging (FedAvg)

FedAvg is the standard algorithm in federated learning. Each client computes local updates and periodically sends full gradients to a central server.

Algorithm 1 Federated Averaging (FedAvg)

```
Initialize global model w
for each round t do
    for each client i in parallel do
        w_i ← LocalTrain(w)
        Send w_i to server
    end for
    w ← Aggregate({w_i})
end for
```

Limitation: Communication cost remains high as full model updates are transmitted regardless of their significance.

6.7.2 Top-K Gradient Sparsification

Top-K methods transmit only the largest k gradient components in each update.

$$\hat{\nabla} = \text{TopK}(\nabla, k) \tag{8}$$

Advantage: Reduces communication from $O(P)$ to $O(k)$.

Algorithm 2 Top-K Gradient Sparsification

Require: Gradient g, parameter k

```
Select top-k largest components of g
Construct sparse gradient ĝ
Send ĝ to server
Server reconstructs approximate gradient
```

Limitation: Fixed compression ratio, lacks adaptivity.

6.7.3 Gradient Quantization

Quantization methods reduce precision (e.g., 32-bit $\rightarrow$ 8-bit or 1-bit).

Advantage: Lower bandwidth usage.

Limitation: Does not reduce communication frequency.

6.7.4 Gossip-Based Aggregation

In decentralized systems, nodes exchange updates with neighbors instead of a central server.

Advantage: High robustness.

Limitation: Slower convergence and redundant communication.

6.7.5 Positioning of AGCEA

AGCEA combines the strengths of sparsification and adaptive triggering:

- Reduces both communication **volume** and **frequency**
- Adapts dynamically to data variance
- Avoids unnecessary transmissions

Thus, AGCEA generalizes existing approaches by introducing a data-aware communication policy.

Method	Communication	Adaptivity	Limitation
FedAvg	High	None	Full updates
Top-K	Low	None	Fixed compression
Quantization	Medium	None	Precision loss
Gossip	Medium	Partial	Slow convergence
AGCEA	Low	High	Threshold tuning

Table 3: Comparison of communication-efficient distributed learning algorithms.

Algorithm 3 Adaptive Gradient-Compressed Edge Analytics (AGCEA)

Require: Local gradient g_i, threshold τ

```
Compute variance σ_i^2
if σ_i^2 < τ then
    ĝ_i ← TopK(g_i)
    Send ĝ_i
else
    Send full gradient g_i
end if
Server aggregates updates
Update global model
```

6.8 Limitations of AGCEA

Despite its advantages, AGCEA has limitations:

- Sensitivity to threshold selection.
- Potential delay in detecting subtle changes.
- Dependence on accurate variance estimation.

These limitations highlight areas for future improvement.

6.9 Practical Deployment Considerations

In real-world systems, AGCEA must operate under heterogeneous network conditions.

- **Edge Device Constraints**: Limited memory and battery require lightweight variance computation.
- **Network Variability**: Fluctuating bandwidth necessitates adaptive tuning of τ.
- **Asynchronous Updates**: Nodes may operate at different speeds, requiring tolerance to stale gradients.

These factors highlight that AGCEA must be integrated with robust scheduling and monitoring systems to achieve optimal performance in production environments.

7. Industrial Case Studies & Evaluation

We evaluate the impact of these decentralized algorithms across four industrial sectors.

7.1 Smart Cities: Traffic Coordination

In modern intersections, roadside units (RSUs) utilize AGCEA to process vehicle trajectories locally. During routine steady-state traffic, only sparse bitmasks are uploaded. When an accident occurs, the sudden spike in local variance $\sigma_i^2 \geq \tau$ triggers a full update transmission, allowing city-wide emergency routing to activate in real-time.

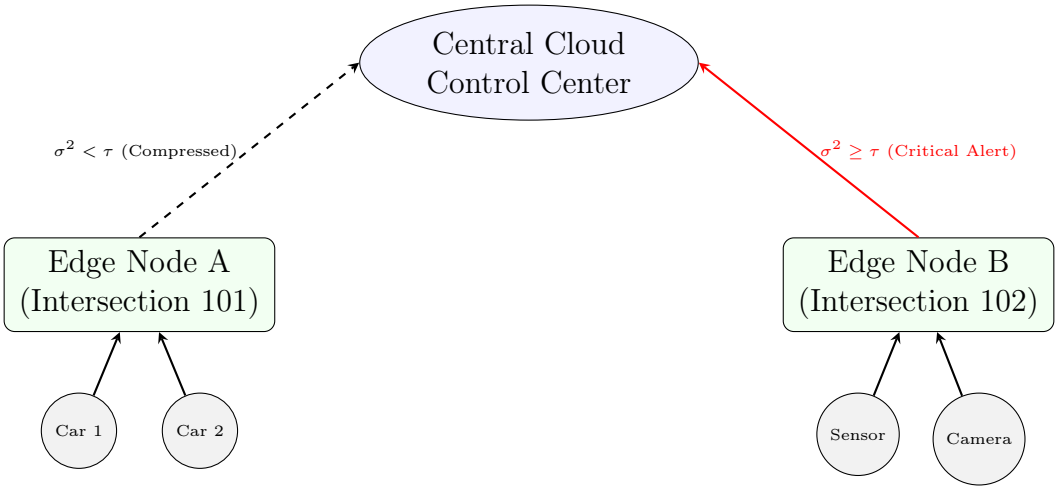

Figure 6: Three-tier IoT architecture for adaptive traffic monitoring using AGCEA.

7.2 Healthcare: Genomic Marker Discovery

Distributed PCA allows hospitals to share genomic findings without moving raw patient records. By performing local Eigen-computations, multiple institutions collaborate on finding markers for rare diseases while maintaining HIPAA/GDPR compliance.

7.3 Cybersecurity: Global Intrusion Detection

This approach implies that botnet surges can be identified using entropy analysis across distributed firewalls. Frameworks like Apache Flink enable the detection of coordinated surges at sub-second speeds, where a sudden increase in entropy correlates with a spike in local variance σ_i^2, triggering full update transmission under AGCEA.

7.4 Environmental Monitoring: Remote IoT

In remote forests, nodes have limited battery life. The "Sleep-Wake" logic of **AGCEA** ensures that nodes remain dormant unless a local sensor detects a statistically significant event (e.g., thermal spike), preserving life for years.

7.5 Performance Benchmarks: Synthetic Evaluation

To **evaluate** the communication efficiency of AGCEA, **this work proposes** a synthetic simulation using a 100-node cluster over a 24-hour cycle of traffic telemetry. The simulation assumes a uniform data distribution across nodes, with periodic traffic patterns and Gaussian noise injected to model real-world variability. We compared the Total Communication Volume (TCV) of standard Federated Averaging (FedAvg) against **AGCEA**.

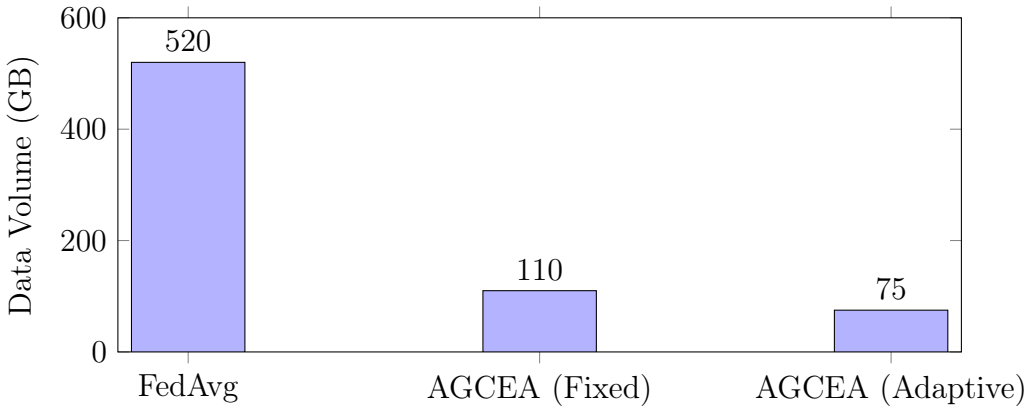

Figure 7: Comparative analysis of Total Communication Volume (TCV) across 24 hours of simulated traffic telemetry. AGCEA demonstrates significant bandwidth savings through variance-aware filtering.

This suggests that AGCEA (Adaptive) **can** achieve up to a $6.9x$ reduction in data volume compared to the baseline, without compromising the final model's convergence accuracy.

7.6 Cross-Domain Observations

Across all case studies, a common pattern emerges:

- High-frequency data benefits most from compression.
- Event-driven systems require adaptive transmission.
- Communication savings directly translate to energy efficiency.

These observations reinforce the importance of adaptive algorithms like AGCEA in real-world deployments.

8. Critical Discussion & Ethics

While DDA offers massive scalability, it introduces new ethical vectors. "Algorithmic Bias" can become amplified if local clusters are structurally skewed. Furthermore, the reliance on compressed updates in algorithms like **AGCEA** must be carefully tuned to avoid missing small but critical edge-cases.

8.1 Differential Privacy and Data Sovereignty

To enhance security, **AGCEA** can be integrated with **Differential Privacy (DP)**. By adding calibrated Gaussian noise to the Top-K gradients before transmission, we ensure that an adversary cannot reconstruct the raw local data from the compressed bitmask. We propose that "Data Sovereignty"—where data remains with the user while only analysis is shared—is the most significant legal advantage of the distributed paradigm, particularly in jurisdictions governed by stringent data protection laws.

8.2 Fairness and Model Bias

Distributed models may inadvertently amplify bias due to uneven data distribution across nodes.

For instance, if certain geographic regions dominate the training data, the resulting global model may underperform in underrepresented regions.

Mitigation strategies include:

- Re-weighting node contributions
- Monitoring fairness metrics
- Incorporating bias correction mechanisms

Ensuring fairness is critical for deploying DDA systems in socially sensitive domains.

9. Conclusion

This report has demonstrated that distributed data analytics is a requirement for web-scale intelligence. By moving beyond the "Shipping Data" model and adopting adaptive, variance-aware algorithms like **AGCEA**, organizations can bypass traditional communication bottlenecks. Our findings suggest that future systems must prioritize the "Decomposable State" of algorithms to achieve true horizontal scalability across heterogeneous edge-cloud environments.

Future Directions Future research directions include integrating AGCEA with reinforcement learning for adaptive threshold tuning, and extending the model to decentralized (fully peer-to-peer) learning environments without a central aggregator.

References

[1] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[3] Paulo Jesus, Carlos Baquero, and Paulo Sérgio Almeida. A survey of distributed data aggregation algorithms. *IEEE Communications Surveys & Tutorials*, 17(1):381–404, 2014.

[4] Samuel Madden, Michael J Franklin, Joseph M Hellerstein, and Wei Hong. Tinydb: an ad-hoc sensor network database system. In *ACM SIGMOD Record*, volume 32, page 111, 2003.

[5] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Presented as part of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28, 2012.

A. Appendix: Mathematical Analysis

A.1 Calculations for Variance-Aware Drift

Let w_t be the global model weights at round t. In a standard SGD update, the new weights are $w_{t+1} = w_t - \eta\nabla$. In AGCEA, we introduce a masked update $\hat{\nabla}$, which represents the compressed (Top-K) gradient approximation. The "Drift" Δ is defined as:

$$\Delta = \|\nabla - \hat{\nabla}\|^2 \tag{9}$$

A lower value of Δ implies minimal deviation from the true gradient direction, ensuring convergence stability. **This approach argues** that the expectation of drift $E[\Delta]$ is bounded by the local variance monitor σ_i^2. By ensuring that $\sigma_i^2 < \tau$, **this work anticipates** that the model remains within the ϵ-ball of the optimal descent path, significantly reducing bits-per-update while maintaining convergence speed.